

VIETNAM NATIONAL UNIVERSITY
BACH KHOA UNIVERSITY
COMPUTER SCIENCE AND ENGINEERING FACULTY



Asia Commercial Bank - Data & Analytic Department

OISP Program - Computer Science Faculty

Internship CO3335 - Semester 243 Report

Supervisor: Nguyen Manh Trung (Vice director of department)
Le Huy Hoang (direct supervisor)
Phan Minh Chien

University Supervisor: Le Binh Dang

Student: Tran Quoc Trung - 2252859

Ho Chi Minh city, October 2025



Contents

1	Introduction	2
2	Foundation Theory	5
2.1	Introduction to Large Language Model	6
2.1.1	Attention mechanism	6
2.1.2	Large Language Models	7
2.1.3	Insight state-of-art LLMs models	9
2.1.4	Agentic LLMs	11
2.2	Retrieval Augmented Generation - LLM-based Chatbot	12
2.2.1	RAG Chatbot workflow	13
2.2.2	Chunking strategy	14
2.2.3	Context Searching	15
2.3	Continuous Integration - Continuous Deployment	16
2.3.1	Containerization	17
2.3.2	Docker	17
2.3.3	CI/CD flow with Docker	18
3	ACB Chatbot Portal	21
3.1	Overview about Chatbot	22
3.1.1	Covered Topic by the Dataset	22
3.1.2	Portal Dataset	22
3.2	Data processing stage	23
3.2.1	Parsing data	23
3.2.2	Chunking & Vectorizing	24
4	Personal Project - Medical Chatbot	26
4.1	Overview about Chatbot	26
4.1.1	Covered Topic by the Dataset	26
4.1.2	Medical Q&A Dataset	27
4.2	System Design	27
4.2.1	Sequence diagram	28
4.2.2	Chainlit Application	29
4.2.3	FastAPI Backend	29
4.2.4	Milvus vectordatabase	30
4.2.5	Langgraph & Langchain Framework	30
4.2.6	Langfuse	32
4.3	Sources	32
5	Conclusion	34

1 Introduction

Asia Commercial Bank (ACB), established in 1993 and headquartered in Ho Chi Minh City, is one of Vietnam's largest and most respected joint-stock commercial banks ACB. ACB's network spans 49 out of 63 provinces and cities nationwide, with around 13,600 employees across 384 branches and transaction offices. Guided by a clear vision to grow sustainably through transformation, ACB's mission emphasizes delivering the best customer experience, achieving high revenue growth, and maintaining a strong Return on Equity (ROE) target of around 20% per annum ACB. ACB's corporate culture rests on five core values: integrity, prudence, innovation, harmony and effectiveness.



Figure 1: Asia Commercial Bank - Main headquarters in 442 Nguyen Thi Minh Khai Street, Ban Co District, Ho Chi Minh City

Along with the explosive growth of Large Language Models and many multi-model in Natural Language Processing fields, since 2024, ACB has embraced AI into their operations as a primary part of revolutionize translation in enhance customer experience, automate internal processes, and strengthen operational efficiency. In the very first stage of AI teams operations, currently there are three kinds of fields that are interesting by ACB:

- **AI-Driven Chatbots:** ACB was among the first banks in Vietnam to roll out advanced AI BOTs featuring intelligent interaction, intent recognition, emotional understanding, and continuous learning to support customer inquiries 24/7. This promises to further improve



customer service.

- **Optical Character Recognition systems:** utilized AI in progress of processing paper document to better and auto extract the texts and detected cheat, illegal overwrite information in PDF.
- **Data collecting and analyzing:** embedding AI in the available workflow of data, such as collecting, analyzing and inferencing data. AI can help to better classifying data and extracting hidden information from data.

Ambition of ACB in AI field is materialized in their environment combines advanced technical infrastructure with a vibrant, people-focused workplace culture. Also, instead of relying on external cloud services for running large language models, ACB has invested in hosting its own dedicated GPU infrastructure, ensuring greater control over performance, security, and operational costs. This in-house capability not only reflects the bank's commitment to technological independence but also enables faster experimentation and deployment of AI solutions. The workplace itself is dynamic and collaborative, attracting many talented young professionals who bring fresh ideas and enthusiasm to innovation projects. For interns, this creates an energizing environment where technical learning goes hand-in-hand with professional growth.

One of the most challenge when working in new AI team in a very big company like ACB is the integration of AI program into their available services. Internship in ACB, not only I have experienced and deepen my knowledge in LLM fields in practical environment, but also I can learn more about the software programming. Throughout my internship, beside the practical knowledge and experience that I have learn from my supervisor, I have chanced to take part in a real Chatbot project, which is called portal Chatbot - a LLM-based Chatbot built in multi-agent framework, that help ACB's staff to better retrieve and look up for portal to make professional decision in their work. Beside that, I practice my learned knowledge to build my simple agentic RAG Chatbot for better understanding about my work. Table 1 show my record for weekly diary of my workload in my 2-month internship.

Table 1: *My internship diary for recording what I have learned in each week.*

Week	Todo List
Week 1	- Learning and presentation overall about LLMs and Chatbot. - Learning about RAG workflow.
Week 2	Building a simple naive RAG system: - Parsing data from PDF files and perform chunking - Build a simple vector database using library chromadb of Python - Test RAG system with question answering
Week 3	- Enhanced RAG system using Reranker model. - Reconstruct code in module, build Chatbot system using Langchain framework.
Week 4	- Build a simple UI for Chatbot using Chainlit library. - Organize Chatbot system in microservice architecture include: Database, Frontend, Backend, Chabot logic, Vector Database.



Week 5	- Building history context for Chatbot. - Upgrade system using LangGraph framework for agentic Chatbot
Week 6	- Wrap the system into Docker. - Integrate system with Langfuse and optimize prompt. - Learning about CI/CD workflow and Kubernetes
Week 7	- Draft the agentic graph logic of system - Optimized prompt for each agent.
Week 8	ACB Chatbot: parsing data using LlamaParser service and optimize chunking strategy
Week 9	- ACB Chatbot: optimize parsing data and optimize prompt. - Personal Chatbot: bulding streaming for answer of Chatbot

This report aim to cover in detailed what I have learned in my internship semester, from theoretical base of Large Language Models and LLM-based Chatbot to software engineering knowledge as well as the practical Chatbot project of ACB and also presented in detailed about my personalized Chatbot product.

The rest of this report is organized as followed: Section 2 will introduce about theory foundation that I have learned within this internship, while in section 3, ACB Chatbot portal project will be presented within some allowed scope due to data security. In section 4, I will describe in detailed about my personal Chatbot application - from theory knowledge to practical experiment. Finally, Section 5 conclude about my experience in this internship in which I will express my overall perceive toward this 2-month experience.

2 Foundation Theory

Natural Language Processing (NLP) is the area of artificial intelligence that enables machines to read, understand, and generate human language. It brings together linguistics, statistics and machine learning to turn raw text and speech into usable representations for tasks like translation, summarization, and question-answering.

A major turning point for NLP was the advent of the Transformer architecture ([6]). Transformers replaced recurrent and convolutional designs with self-attention mechanisms that scale much better in parallel computation and capture long-range dependencies in text. This architectural change - combined with two practical advances, large-scale self-supervised pretraining on massive text corpora and the engineering ability to train models with billions (or more) of parameters led directly to today's Large Language Models (LLMs). By learning broad statistical patterns of language from pretraining and then adapting via fine-tuning or instruction tuning, LLMs achieve strong zero-/few-shot performance across many tasks without task-specific architectures.

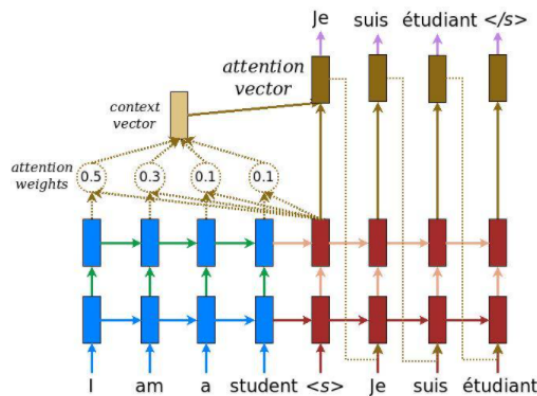


Figure 2: *Attention mechanism - heart of transformer architecture*

LLMs today already cover a wide and growing set of capabilities; likely directions and near-term developments include: zero- or few-shot generalization, generating of human natural conversation, multi-agent workflow and so on... . Many application of LLMs can now be widely supported by state-of-art GPU infrastructure and cloud services, which allow it's deployment and integration with wide range of Software application.

One of the most popular application of LLMs is the use of LLM into Chatbot system. The integration of Large Language Models (LLMs) into chatbot systems has significantly transformed their capabilities, surpassing the limitations of traditional rule-based chatbots. Unlike rule-based systems, which rely on predefined scripts and rigid keyword matching, LLM-based chatbots can understand context, handle diverse user inputs, and generate human-like, contextually relevant responses in real time. They excel at managing ambiguous or open-ended queries, adapting to new topics without manual rule updates, and following complex multi-turn conversations. Moreover, through techniques like Retrieval-Augmented Generation (RAG) and instruction tuning, LLM-powered chatbots can combine real-world knowledge with conversational fluency, making them far more flexible, scalable, and engaging for both customer service and internal support scenarios.

On the other hand, integrating chatbot applications into modern software engineering workflows requires not only AI expertise but also the discipline of established engineering practices, particularly

Continuous Integration and Continuous Deployment (CI/CD). While the conversational AI component may be novel, the value of software engineering fundamentals remains unchanged—version control, automated testing, modular architecture, and deployment pipelines are as essential here as in any other software domain. By embedding chatbot development into a CI/CD process, teams can automate the building, testing, and deployment of updates, ensuring that new features, improved AI models, and bug fixes are delivered quickly and reliably. This approach preserves code quality, reduces integration risks, and enables rapid iteration, allowing chatbot systems to evolve in step with user needs and business requirements without sacrificing stability or maintainability.

In this section, I aim at presenting an overview for theory foundation of LLMs, Chatbot system and also introduce to software engineering terminology that I have learned during my trip. The rest of this section is organized as following: subsection 2.1 introduce about LLM background and aim to explain it's revolution against traditional natural language processing field. Then, a workflow for a RAG system of LLM0-based Chatbot are presented in subsection 2.2. Finally, a piece of software engineering field CI/CD workflow will be presented in subsection 2.3.

2.1 Introduction to Large Language Model

Natural Language Processing is a traditional machine learning-based field that began with clearly defined tasks—tokenization, part-of-speech tagging, named-entity recognition, parsing, machine translation, sentiment analysis, and text classification—often solved with feature engineering and then with statistical sequence models such as Hidden Markov Models and later Recurrent Neural Networks (RNNs) and LSTM/GRU variants that could model temporal dependencies in text. The appearance of RNNs with hidden state first introduce Deep neural network into NLP by maintaining the semantic capture of a long sequence of text. However, they suffered from fundamental bottlenecks: sequential (time-step) computation that limits parallelism on modern hardware, difficulty learning very long-range dependencies, and training instabilities (vanishing/exploding gradients), all of which made scaling to very large datasets and model sizes inefficient.

In 2017, in [6], Vaswani et al. introduce attention mechanism, which allow the model to process tokens parallelly and solve the problem of gradient. Attention mechanism marked the revolutionize in development of Language Models. The appearance of Transformer architecture family allow the scalability from those language models which can only do the simple task on natural text such as classification or plagiarism detection into those Large Language Models with billions number of parameters, which can cover wide range of NLP task with deep understanding of human natural language semantic. Figure 3 visualized the development of LLMs models by year since the introduce of Transformers.

2.1.1 Attention mechanism

The core of modern transformer models is the *scaled dot-product attention* mechanism. Formally, given the matrices

$$\mathbf{Q} \in \mathbb{R}^{m \times d_k}, \quad \mathbf{K} \in \mathbb{R}^{n \times d_k}, \quad \mathbf{V} \in \mathbb{R}^{n \times d_v},$$

A Brief History of LLMs

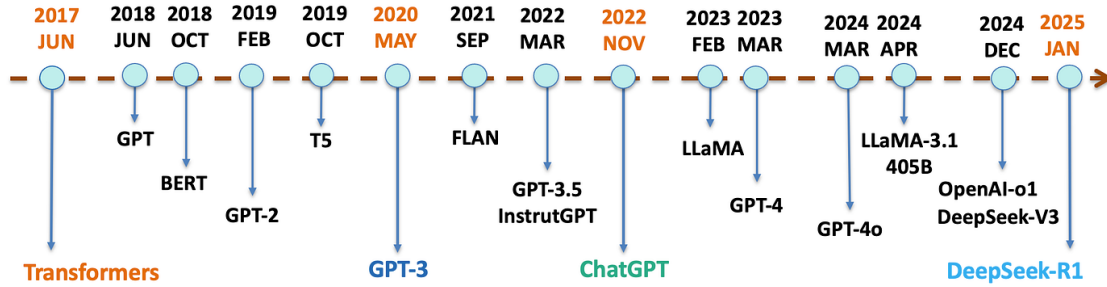


Figure 3: A brief history of LLMs development

The attention output is computed as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

The attention mechanism functions much like a reader who, upon examining a particular word (represented by the Query vector $\mathbf{Q}$), scans the entire sentence to find which other words (each represented by a Key vector $\mathbf{K}$) are most relevant to understanding it. The similarity between $\mathbf{Q}$ and each $\mathbf{K}$ is measured via a dot product, producing attention scores that reflect how strongly each word should contribute. These scores are scaled (to stabilize gradients), converted into probabilities via softmax, and then used to compute a weighted average of the corresponding Value vectors $\mathbf{V}$. The result is a context-aware representation of the queried word, informed by its relationship to every other word in the sequence. This mechanism enables models to dynamically “focus” on relevant context when interpreting or generating each token. Additionally, unlike RNNs which process tokens sequentially and suffer from issues like vanishing gradients and slow computation, attention allows simultaneous (parallel) computation across all tokens and direct access to long-range dependencies, thus improving both efficiency and modeling power.

This mechanism enables full parallel processing of all token positions—unlike RNNs, which process each token sequentially, suffering from slow training and gradient instability. By allowing each token to directly “attend” to all others, attention overcomes the limitations of RNNs in scalability and long-range dependency modeling.

2.1.2 Large Language Models

The debut of the Transformer in 2017—with its revolutionary attention mechanism—sparked a surge of interest among researchers in the capabilities of language models. Shortly thereafter, landmark models like GPT-1 ([4]), GPT-2 ([5]), and BERT ([2]) emerged, each containing hundreds of millions of parameters and achieving impressive results on standard benchmarks. However, these early models primarily focused on incrementally enhancing performance on traditional tasks—such as classification, translation, or text generation—without probing the deeper semantic potential that scaling might reveal. Their scope remained largely bounded by supervised fine-tuning. The landscape shifted dramatically with the advent of GPT-3 ([1]) in 2020, scaling

up to an unprecedented 175 billion parameters. This massive model demonstrated emergent abilities in few-shot, zero-shot, and in-context learning, signaling a new era where sheer scale opened avenues for more flexible, general-purpose language understanding. Thus, while earlier architectures laid the groundwork, the field of Large Language Models truly accelerated and captured widespread attention following the scalable leap made by GPT-3.

Emergent Ability of Large Language Models

From the Language Models, along with the scaling up of number of parameters, the performance on the benchmark of models are also scaled in the simple task such as word prediction, classification task,... , all can be stated as **Scaling Law**. Meanwhile, **emergent abilities** of LLMs are formally defined as “the abilities that are not present in small models but arise in large models” ([8]), which is the most prominent features that distinguish LLMs from previous language models. Those **emergent abilities** that can be introduced by those LLMs models nowadays include:

- *In-context learning*: The ability of a model to perform a task by conditioning on examples provided in the input prompt—without any gradient updates—allowing it to adapt instantly to new tasks with only a few examples or even none (zero-shot).
- *Instruction following*: The capacity to understand and execute tasks described in natural language instructions, even for instructions never seen during training, enabling more natural and flexible human–AI interaction.
- *Step-by-step reasoning*: The capability to decompose complex problems into intermediate steps, often expressed as chain-of-thought reasoning, which improves performance on tasks requiring logical deduction, multi-hop reasoning, or mathematical problem solving.

The process of using any Language Models including two steps: pretraining stage - where model are trained from scratch to understand the semantic inside natural language, and fine-tuning - stage of adapting model on downstream task to achieve high performance on a specific task.

Pretraining stage

Pretraining prefer to the initial stage in which a language model is trained on massive amounts of unlabeled text data to learn general linguistic, semantic, and contextual representations. The quality and diversity of the pretraining dataset directly influence the model’s abilities: models trained on code-rich corpora (e.g., GitHub repositories) exhibit stronger programming skills, while datasets containing books, academic articles, and multilingual resources enhance narrative fluency, domain-specific knowledge, and cross-lingual competence. Dataset preprocessing—such as tokenization, deduplication, and filtering for quality—plays a critical role in shaping the model’s performance across these domains. Furthermore, the decision regarding the number of model parameters is closely linked to the breadth of pretraining data: larger models typically require more extensive and diverse datasets to fully exploit their representational capacity without overfitting.

During **pretraining**, the learning objective is typically framed as a self-supervised task designed to capture the statistical dependencies in natural language. For autoregressive models (e.g., GPT family), the task is *next-token prediction*, where the model is trained to generate the next word in a sequence given all previous words. In contrast, masked language models (e.g., BERT) use the

masked word prediction objective, in which a fixed proportion of tokens in the input sequence are replaced with a special [MASK] token, and the model is trained to recover the masked tokens from the surrounding context. This bidirectional conditioning enables the model to capture information from both left and right contexts. Other variations, such as span masking (used in T5), randomly mask contiguous spans of text instead of single tokens, encouraging the model to understand larger semantic units. The choice of pretraining objective directly impacts the representation quality and the downstream adaptability of the model.

Finetuning

Finetuning is the subsequent adaptation of the pretrained model to specific downstream tasks, enabling it to meet targeted performance requirements. Different finetuning strategies vary in efficiency and objective:

- *full-parameter finetuning*: updates all model parameters, offering high adaptability but at significant computational cost
- *parameter-efficient tuning* methods—such as LoRA, adapters, or prefix tuning—modify only a small subset of parameters, making adaptation more resource-efficient while preserving most of the pretrained knowledge.
- *Alignment tuning* focuses on making model outputs safer, more useful, and more aligned with human intent, often leveraging **reinforcement learning from human feedback (RLHF)**.

In RLHF, human annotators evaluate model responses, and these preferences are used to train a reward model, which subsequently guides the LLM through reinforcement learning. This process ensures that the model not only performs well on its target task but also behaves in a way that matches human expectations and ethical constraints.

2.1.3 Insight state-of-art LLMs models

The trend in those recent years on LLMs is not focusing on increasing the number of parameters on models, but focus more on commercial aspect of LLMs, including the throughput of models and the cost efficient on large model's operation - hosting, training and inference. Although as **scaling law**, LLMs model can perform more flexibility task with higher performance. We may think about the downside of increasing the number of parameters, including:

- Higher cost for hosting and training.
- The slow down in inference time. Deeper layers mean the processing time also increase.

Those problems are also the interested by many LLMs researchers. Among them, I am interesting in two concepts: Mixture of Expert architecture and Distillation Knowledge. Although there may be more and more works on this fields, due to the limitation scope of my report, I only present about Mixture of Expert architecture and Distillation Knowledge. Finally, a study case about a state-of-art model - Qwen3 are also presented in this subsection.

Mixture of Experts (MoE) Architecture

Mixture of Experts (MoE) is a sparse-activation architecture designed to increase model capacity without a proportional increase in inference cost. An MoE layer contains a set of expert subnetworks $\{E_1, E_2, \dots, E_M\}$ and a lightweight gating network $G(\cdot)$. For each input token (or token representation) x , the gate computes routing scores $g(x) \in \mathbb{R}^M$; typically the top- k highest-scoring experts are selected and the final MoE output is a weighted combination of only those experts' outputs. Formally, if $\mathcal{S}(x)$ denotes the indices of the top- k experts and $p_i(x)$ the normalized gating weight for expert i , the MoE layer produces

$$\text{MoE}(x) = \sum_{i \in \mathcal{S}(x)} p_i(x) E_i(x).$$

The key benefit of MoE is *conditional computation*: although the model contains a very large total number of parameters across all experts, only a small subset is activated per token. This yields an effective increase in representational capacity (many parameters) while bounding per-token compute (proportional to k). In practice, this enables models to scale to extremely large parameter counts (and hence richer representations) without linearly increasing FLOPs for every inference step.

However, MoE designs introduce practical and algorithmic challenges. Efficient training and serving require careful distributed coordination because experts may reside on different devices; naive routing can cause communication bottlenecks and uneven expert utilization (so-called expert collapse). Common engineering remedies include top- k gating, auxiliary load-balancing losses that penalize uneven expert usage, capacity factors to limit per-expert load, and noisy gating to promote exploration. Despite these complexities, MoE variants (e.g., Switch-style and GShard-style designs) have demonstrated that sparse expert routing is a powerful tool for trading off total parameter capacity against per-token computation and cost—making MoE a compelling approach for building high-capacity yet cost-aware LLMs in commercial settings.

Knowledge Distillation

Knowledge distillation ([3]) is a model compression technique in which a large, high-performing "teacher" model transfers its knowledge to a smaller "student" model so the student can achieve comparable performance with far fewer parameters and lower inference cost. Concretely, distillation commonly minimizes a combination of the standard task loss on ground-truth labels and a *distillation loss* that encourages the student to match the teacher's softened output distribution. A frequently used formulation is the temperature-scaled Kullback–Leibler loss:

$$\mathcal{L}_{\text{KD}} = \tau^2 \text{KL}(\text{softmax}(z^{(t)}/\tau) \parallel \text{softmax}(z^{(s)}/\tau)),$$

where $z^{(t)}$ and $z^{(s)}$ are the teacher and student logits and τ is the temperature. Beyond logit (soft-label) distillation, practical variants include *feature distillation* (matching intermediate representations), *attention-map distillation*, and *sequence-level* distillation for generative tasks (where the teacher generates synthetic targets to train the student). Modern workflows often combine distillation with other compression techniques—quantization, pruning, and parameter-efficient

tuning—for further efficiency gains.

Distillation is particularly valuable when deploying language models in production: it reduces memory footprint and latency, lowers hosting and serving costs, and enables on-device or edge inference. Nevertheless, distillation poses challenges: a large capacity gap can limit what a small student can learn, important emergent behaviors (e.g., complex chain-of-thought reasoning) may not transfer easily, and distilled models can inherit biases or calibration issues from their teachers. Careful choice of teacher, distillation objective (logit vs. feature vs. sequence-level), and data (including teacher-generated augmentation) is therefore critical to produce a compact model that preserves the teacher’s useful behaviors for downstream applications.

Study case: Qwen3

Qwen3 ([7]), the third-generation Large Language Model family from Alibaba, distinguishes itself with exceptional multilingual prowess and an adaptable suite of models tailored for diverse deployment needs. Exceeding its predecessors, Qwen3 supports an impressive 119 languages and dialects, enabling robust cross-lingual understanding and generation across global linguistic contexts. In addition, the Qwen3 series offers remarkable flexibility in model architecture and size. The dense model variants range from compact 0.6B up to 32B parameters, ideal for mobile or edge applications, while the Mixture-of-Experts (MoE) versions scale up to 235B total parameters—though only 22B are activated per inference step for efficiency. This spectrum of model sizes empowers developers to select the optimal balance between performance and resource utilization. Collectively, Qwen3’s linguistic breadth and scalable architecture underscore its strength as an open-source, globally oriented LLM platform. Those small models are regarded as distillation version of big models. Here is list of family of models that are supported:

- Qwen3-8B - dense models.
- Qwen3-14B - dense models.
- Qwen3-32B - dense models.
- Qwen3-30B-30A - Mixture of Expert models.
- Qwen3-235B-A22B - Mixture of Expert models.
- So on... .

2.1.4 Agentic LLMs

While the traditional LLMs process the incoming request consecutively in a single turn to generate response, this make traditional LLMs work very poor in those reasoning or complex task. An approach for this is to used LLMs in an agentic framework, where each agent act as a single independent LLMs with their own tasks and instruction; they exchange their working and decompose the request into smaller stages of thinking and hence perform better on complex task. See Figure 5.

In real-world systems, agentic LLMs have demonstrated significant advantages in domains where reasoning depth, adaptability, and integration with external systems are essential. For example, they are applied in *autonomous research assistants* that can search literature, analyze findings,

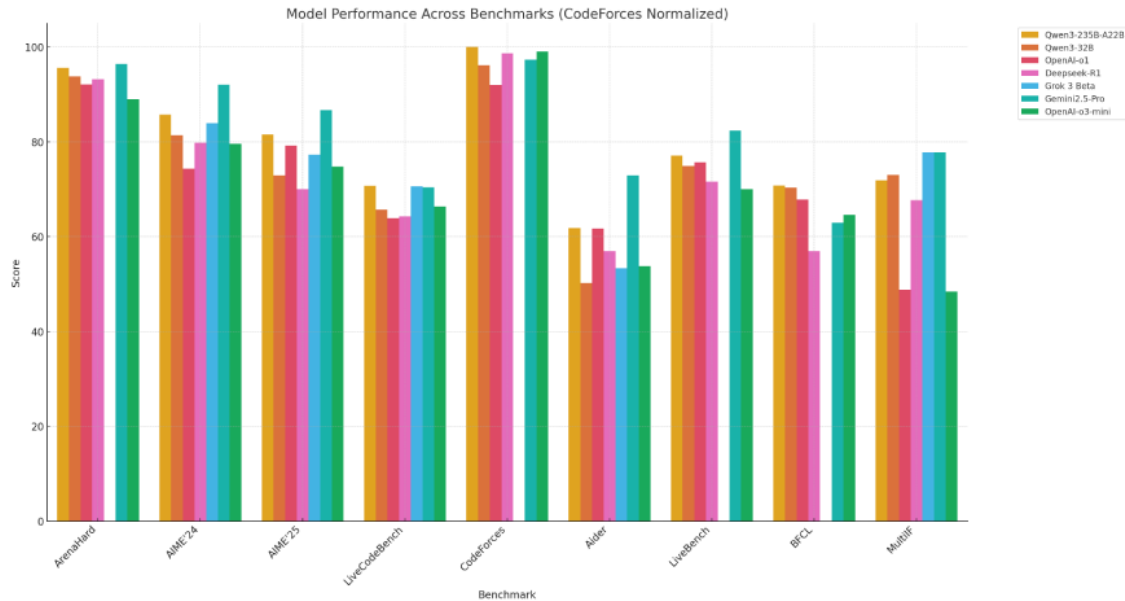


Figure 4: Performance of Qwen3 models across different benchmarks

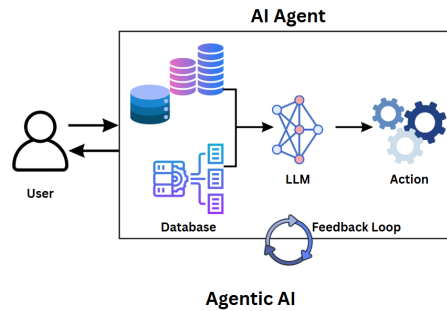


Figure 5: Agentic Framework

and synthesize structured reports; in *business process automation* where they orchestrate workflows across multiple enterprise systems; and in *software development* to manage multi-file code generation and debugging iteratively. By moving beyond single-turn prediction and embracing an interactive, tool-augmented reasoning process, agentic LLMs provide a pathway to deploying language models in complex, production-grade applications.

2.2 Retrieval Augmented Generation - LLM-based Chatbot

One of the most interesting application of LLMs is the field of Chatbot, which is application that can simulate human conversation through text or voice interactions. Chatbot can serve a huge range of usages, including providing the customer services, support making decision in professional work or help better retrieve data,... . With the natural ability when being pretrained on huge dataset, many Chatbot can not ensure the verification of the information that they produce and suffer against **hallucination**, a phenomenon where AI systems, particularly large language models (LLMs), generate outputs that are incorrect, nonsensical, or lack a factual basis, while still appearing plausible.

One of the way to control this is instructing LLM model where to look up for the information, i.e, provide them with context for their response generation. We may recall from the earlier

section 2.1.2 of this report that this is the "in-context" emergent ability. Here is where Retrieval Augmented Generation take place in Chatbot application. The role of this system is too auto retrieve the *uitable* context data from database and auto feed it into the query to the Chatbot. See Figure 6.

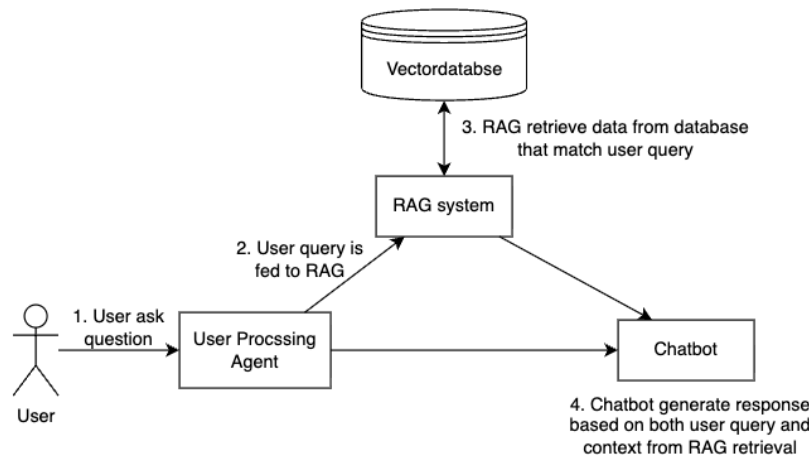


Figure 6: *Integration of RAG into Chatbot*

This subsection focus on presenting the core idea and challenges in RAG Chatbot system. The content is organized as followed: Section 2.2.1 present about detail workflow of a RAG Chatbot system; Section 2.2.2 and 2.2.3 focus on main challenges in RAG: context retrieval by analyzing the processing on background data. Finally, section ?? discuss approach of applying multi-agent in RAG-Chatbot.

2.2.1 RAG Chatbot workflow

The workflow of a RAG Chatbot system can be generally divided into two main phases: **data processing** and **data inference** (see Figure 7. These phases ensure that the Chatbot can effectively retrieve and utilize the most relevant context for generating accurate answers. The detailed steps are as follows:

1. Data processing:

- (a) Parse the input data from various sources to ensure it is in a clean and consistent format.
- (b) Split the data into smaller, semantically meaningful chunks and attach corresponding metadata (e.g., source, timestamp, category) for better indexing and retrieval.
- (c) Generate vector embeddings for each chunk using an embedding model and insert them into a vector database for efficient similarity search.

2. Data inference:

- (a) Generate the vector embedding of the user's query using the same embedding model as in the data processing phase.
- (b) Perform semantic search on the vector database to retrieve the most relevant chunks of information.

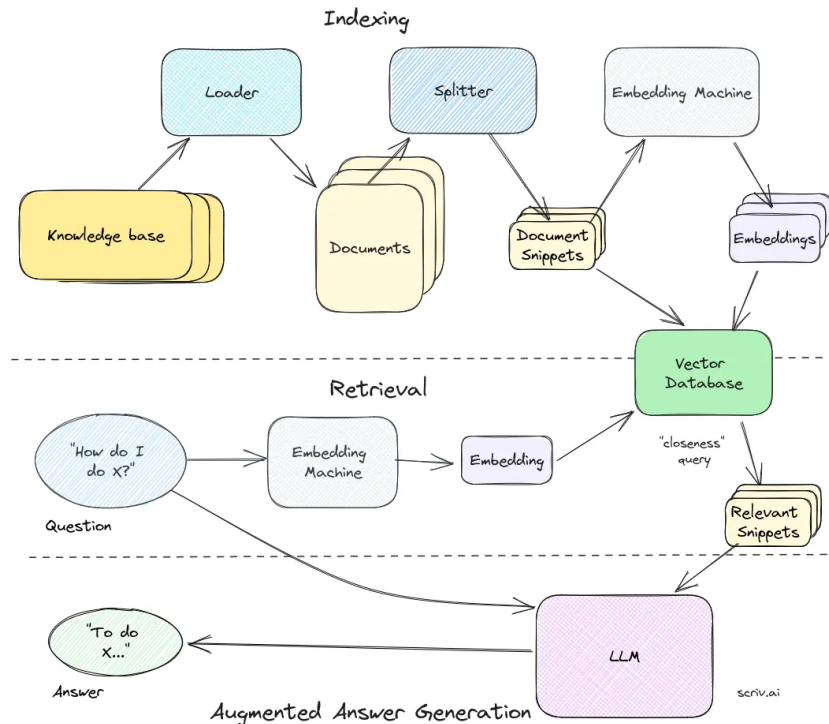


Figure 7: Workflow of RAG-based Chatbot

- (c) Re-rank the retrieved chunks using a dedicated reranker model to ensure that the most contextually suitable chunks are prioritized.
- (d) Feed the top-ranked chunks, along with the user query, into the Chatbot model to generate a final, context-aware answer.

2.2.2 Chunking strategy

The quality of Chatbot answering depended heavily on the context given by RAG model. The problem of retrieving context is one of the main challenges in RAG Chatbot in an optimization problem: RAG system must travel through all of data in database and look up for those piece of data that are likely to contain the expected context that can be used for answer question. The context length of input for Chatbot depend heavily on length of context provided by RAG. Chunking strategy provide a mechanism for splitting the data into smaller part, called chunk, which is the unit of searching and processing in Chatbot context. A good chunking strategy will ensure that:

- Each chunk do not content multi-content information and do not too long, which will make the input for Chatbot model too long and may affect context length of model.
- Each chunk should not too short, which may increase the size of database. Each chunk will be represented as a single object in vector database.
- A mechanism for adding metadata to each chunk.

There are 5 levels of chunking strategy:

• Fixed size chunking

This is the most crude and simplest method of segmenting the text. It breaks down the text into

chunks of a specified number of characters, regardless of their content or structure. It do not care about semantic of each chunk and only chunk the data when number of character reach threshold. This approach break down the semantic of data chunks and provide very poor semantic vector search.

- **Recursive Chunking**

In this method, we divide the text into smaller chunk in a hierarchical and iterative manner using a set of separators. If the initial attempt at splitting the text doesn't produce chunks of the desired size, the method recursively calls itself on the resulting chunks with a different separator until the desired chunk size is achieved by list of separator such as " "; "\n"; "\n\n";

While it perform better at Fixed size chunking, it's can not ensure the size of each chunk and still capture worse the semantic structure of data.

- **Document-based Chunking**

In this chunking method, we split a document based on its inherent structure. This approach considers the flow and structure of content but may not be as effective documents lacking clear structure.

- **Document with Markdown:** Langchain provides MarkdownTextSplitter class to split document that consist markdown as way of separator.
- **Document with tables:** When dealing with tables, splitting based on levels 1 and 2 might lose the tabular relationship between rows and columns. To preserve this relationship, format the table content in a way that the language model can understand (e.g., using `<table>` tags in HTML, CSV format separated by ';', etc.). During semantic search, matching on embeddings directly from the table can be challenging. Developers often summarize the table after extraction, generate an embedding of that summary, and use it for matching.
-

- **Semantic Chunking**

This chunking method aims to extract semantic meaning from embeddings and then assess the semantic relationship between these chunks. The core idea is to keep together chunks that are semantic similar. This method ensure the semantic of each chunk and hence gain very high performance on context retrieving.

- **Agentic Chunking**

This chunking strategy explore the possibility to use LLM to determine how much and what text should be included in a chunk based on the context. To generate initial chunks, it uses concept of Propositions based on paper that extracts stand alone statements from a raw piece of text. Langchain provides propositional-retrieval template to implement this.

After generating propositions, these are being feed to LLM-based agent. This agent determine whether a proposition should be included in an existing chunk or if a new chunk should be created. This is the most expensive strategy and can deal with very complex unstructured data.

2.2.3 Context Searching

Searching prefers to the process in inference time that look up for those chunks that are likely to contain the context information that can be used for answer user questions. Searching strategy is

a two-dimensional optimization problem:

1. **Precise:** correctly search for those chunk that may contains the needed information and reduce the wasted chunk that can affect the context length of mdoel.
2. **Searching time:** among the dataset with very large number of chunks, the time complexity of searching maybe the bottleneck of whole Chatbot workflow.

The second constraints make the option of using LLMs to search for whole database is impossible because feeding all database into LLMs model make the time complexity explore. Hence, as presented in the RAG workflow in Figure 7, the searching problem is a two-stage process including two steps: semantic search (or so-called vector similarity search) and reranking.

• Semantic search

After chunking, each chunk is saved in vector database with their vector embedding. In the inference time, at sematic search step, the query from user is embedded into vector and we apply vector similarity to filter top-N chunks that are most similar to the query vector. Compared to use LLM, vector similarity is a **lightweight process** of filtering in the first level.

However, in some specific topic, for example in bank dataset, some chunks may share the same semantic vector although they indicate the difference document type. That is the main disadvantage of embedding search. A solution is adding a sparse vector for keyword matching to each chunk vector, i.e, by using TF-IDF algorithm.

Once again, the choose of embedding vector is flexibility for specific fields and can be used to implement soft-assign filter for metadata in chunks. The output of this stage is top k number of documents - a number that is extremely less than total documents but still violate the context window of Chatbot.

• Reranker

Reranker prefer to the second stage of chunks retrieval, where each chunk is evaluated using LLMs. Rerankers are those lightweights Language Models that focus on scoring the semantic of a sequence of texts. The query is user is respectively concatenated with chunk before scored by reranker models.

2.3 Continuous Integration - Continuous Deployment

Continuous Integration/Continuous Deployment is a concept in Software Engineering that refer to the automated pipelines that continuously integrate code changes, run automated builds and tests, and push verified artifacts toward production-like environments so that software can be released reliably and frequently. Continuous Integration emphasizes frequent merges and automated verification to catch integration defects early, while Continuous Delivery/Deployment automates the promotion of built artifacts through staging to production (with human gates for delivery or fully automated deployment). Implemented with version control hooks, build agents, artifact registries, infrastructure-as-code, automated tests, observability and rollback strategies, CI/CD reduces integration risk, shortens feedback loops, and makes releases reproducible and auditable.

CI/CD is one of the cores in field of Software Engineering, which we have been introduced in course Software Engineering (CO3001). In this report, I focus more on the core idea of this flow by studying concept of **Containerization** in Section 2.3.1 and learn about Docker - a containerization

platform, in Section 2.3.2. Finally, a CI/CD that work with Docker image are represented in Section 2.3.3.

2.3.1 Containerization

The basic concept of CI/CD is the **containerization** terminology, which is the process of packaging an application together with its runtime, libraries and dependencies into a portable, reproducible unit, eliminating “works-on-my-machine” problems and ensuring consistent behavior across development, test and production environments. Reminds in Operation system course (CO2017), virtualization is another mechanism that enable multiple workloads to run on a single physical server. However, they are significantly differed in their implementation ways, Virtualization creates full virtual machines (VMs), each with its own operating system, while containerization packages applications with their dependencies into lightweight, isolated containers that share the host OS kernel.

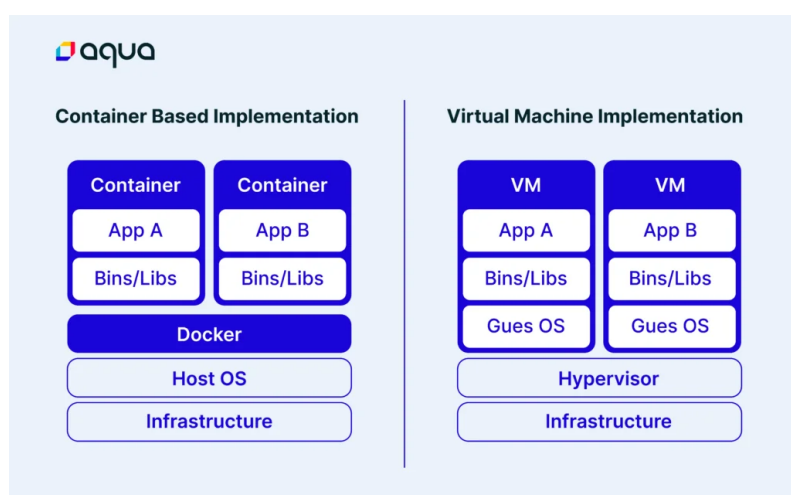


Figure 8: In Container implementation, all containers run on the same Host OS, via the intermediate management of Docker platform; they share the resource of computer (CPU & GPU) while maintain their own environment independently. While in virtual machine, each VM run on their own OS and deeply independent in their computational and memory resource.

2.3.2 Docker

Docker is the most popular containerization platform that can produce immutable, versionable container images and a layered filesystem, Docker supplies standard deployable artifacts with fast startup and efficient resource isolation. In CI/CD pipelines Docker enables repeatable build-and-test stages, parallelizable integration tests, image registries for artifact storage/versioning, and automated deployment/rollback steps—streamlining delivery and improving release reliability.

Docker provides the ability to package and run an application in a loosely isolated environment called a container. By isolation mechanism, Docker allow user to run simultaneously many process in docker. Containers are lightweight processes that contain everything a process need to be run, including all setup environment. The main strength of Docker is the ability to package processes into script and can build/run a Docker container process only using its the script. This allow Docker to be a powerful tools for managing flow of CI/CD as well as sharing a program across

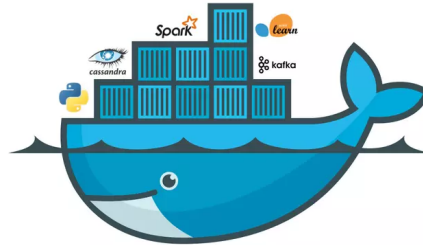


Figure 9: *Docker - lightweight containerization platform*
users in different computer environment.

Before study more about mechanism of Docker, I want to introduce some important terminology that refer to Docker objects:

- **Docker container:** isolated processes for each of application's components. All of the environment, running logic, dependencies,... of a process are package and run within a container.
- **Docker images:** a read-only blueprint that defines everything needed to run an application inside a container. A Docker images can be refer as a static program of a container process that contain static configuration without dynamic runtime environment.
- **Docker volumes:** a mechanism for persisting data generated and used by Docker containers. When a container stops or is removed, all of its internal data is lost because containers are ephemeral. Volumes solve this problem by providing a way to store data outside the container's lifecycle.
- **Docker network:** a way to enable communication between containers, the host system, and the outside world. Each container can see and communicate with other container in the same network.
- **Docker compose:** a tool that lets user define and manage multi-container applications using a single configuration file.
- **Docker hub:** a cloud-based registry service where you can store, share, and distribute Docker images. It works like GitHub, but for container images.

Then, the working flow of creating and running an application in Docker containers:

1. Write a configuration file (using `docker-compose.yml` for docker compose or `Dockerfile` for single container). This file specify how Docker images should be built or installed from Docker hub. Configuration file also allow user to config docker volume and docker network for those containers.
2. Establishing Docker images from configuration files.
3. Execute docker images to run docker container.

2.3.3 CI/CD flow with Docker

Continuous Integration (CI) is the first stag in CI/CD flow, which ensures that code changes contributed by different developers can be reliably integrated into a shared repository without breaking the system. At this step, each commit or pull request is automatically built and verified through a sequence of checks, including compilation, unit testing, static code analysis, and style validation. The detailed flow of Continuous Integration steps including:



1. Developer commit code change to a Git-based platform, typically Github or GitLab and create merge request for merge to main branch. This will trigger the flow of CI.
2. The management agents, such as ArgoCD build docker image from new code and run docker container for project to run in development environment.
3. The application is then be run in unit testing in environment to ensure verify if there is no error in new version.
4. As soon as the application pass unit testing, Docker images of the application are pushed into private Docker hub.

Continuous Deployment (CD) refer to the second stage of CI/CD flow that automate the release of verified build artifacts into staging or production environments, ensuring that software updates can be delivered quickly and consistently. The primary role of this step is to eliminate manual deployment overhead by using automated scripts and pipelines that handle tasks such as packaging applications, provisioning infrastructure, and rolling out updates with minimal downtime. Technically, CD leverages container orchestration platforms such as Kubernetes or Docker Swarm, infrastructure-as-code tools like Terraform or Ansible, and deployment strategies including rolling updates, blue-green deployments, or canary releases. By integrating these technologies, Continuous Deployment guarantees that Docker images or other build artifacts are delivered in a reliable, reproducible, and scalable manner. This automation not only accelerates the release cycle but also provides mechanisms for rollback and monitoring, ensuring that new features reach users rapidly without compromising system stability. The detailed flow of Continuous Deployment including:

1. After CI flow, a new Docker image is pushed into Docker hub. There are two choices to trigger CD flow: first way, developer update image name inside configuration file and commit this change into GitOps repo, require the acceptance of DevOps to trigger CD flow; second way, the CI/CD flow work continuously in which new Docker Image are triggered in CD flow.
2. The management agents, such as ArgoCD is triggered and compare the change in configuration file. It auto decide which update, delete or build action must be made next.
3. ArgoCD agents make change to Kubernetes cluster and complete the CI/CD flow.

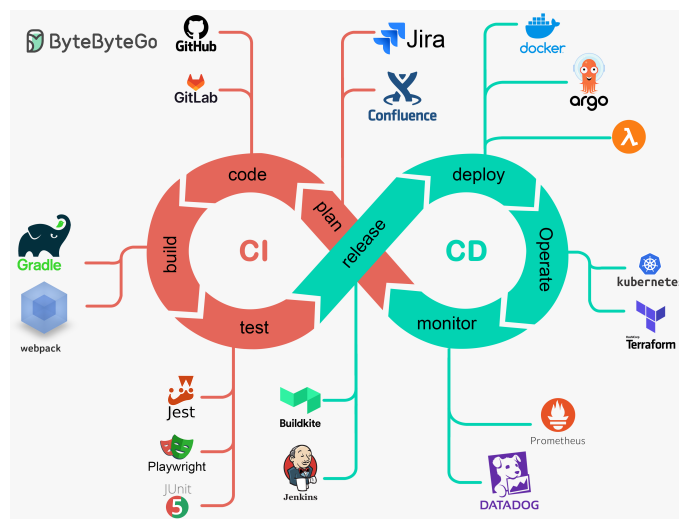


Figure 10: *CI/CD workflows map: from change commit in code to deployment and update in whole system*

3 ACB Chatbot Portal

Nowadays, along with the development of LLMs and AI with those state-of-art models, which allow computer science to achieve huge performance and gain noticeable consideration from all big company. ACB become one of the pioneering banks to utilized AI in upgrading their overall system and service with the main focus on the first stages are OCR and Chatbot fields. Especially in LLM domainm ACB has been provided and golived a Chatbot application for customers, or we may called **External-user Chatbot**, focus on supporting **Customer service representative** (CSR) task. Reader may find and use this Chatbot from official mobile application of ACB: [ACB ONE Biz](#).

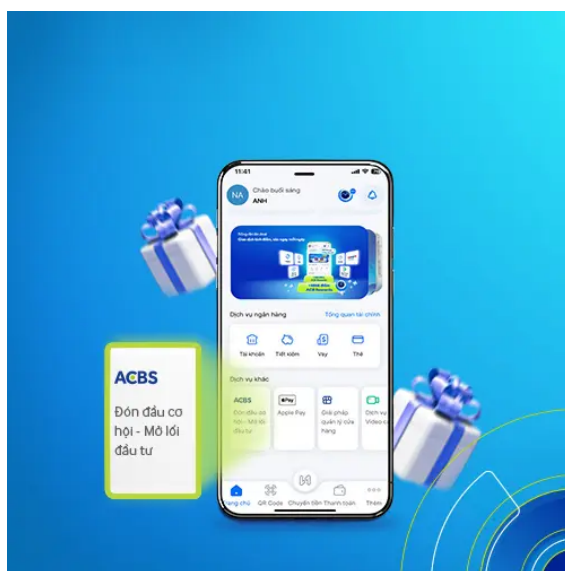


Figure 11: Main pages of ACB One Biz mobile application

Chatbot Portal is one of the Chatbot systems that ACB's AI Teams currently working on the first version. Different from **External-user Chatbot**, **Chatbot Portal** focus on serving ACB internal staffs for work will ACB official documents. The main work in this project cover a wide range of computer fields, from Chatbot-based technology to DevOps and Software knowledge. As AI Engineers, our teams work mainly from the scope of data science task, developing Chatbot-application up to scope of stages Continuous Integration (the theory can be reviewed in Section 2.3).

My main task in this project refer to data processing stage, from the raw data in PDF formats to chunks to be loaded into Vector database. In this section, my focus is covered what I have learned in my internship from this project. Hence, this section will covered two main contents: subsection 3.1 present about overview of ACB Chatbot including the main functionality & a brief review about dataset; finally, subsection 3.2 discuss about my contribution in this project - Data processing.

Due to the privacy policy in ACB, I can not upload or provide any detailed content as well as statistic of our works in this report.

3.1 Overview about Chatbot

The developed chatbot is designed as a specialized Banking Assistant with the primary objective of supporting ACB staffs in retrieving accurate information about business activities within Asia Commercial Bank (ACB) with their source. From the development perspective, the development of this process can be summarized as stages:

1. **Prepare data:** determine the scope covered by Chatbot to prepare the list of PDF files from ACB Portal system and prepare metadata for each file.
2. **Data processing:** parsing PDF data into Markdown types. Then perform chunking on each documents, concatenate with their corresponding metadata and load data into vector database system.
3. **Development of Chatbot workspace:** develop Chatbot application in workspace.
4. **Optimization:** Design Multiagent graph for handle variety range of request and optimizing prompt for each agent.
5. **Continuous Integration:** containerize application, build image and do unit testing for system.
6. **Continuous Deployment:** upload image into ACB's private Docker hub, trigger the configuration file of CD workspace and perform update on Kubernetes cluster.

3.1.1 Covered Topic by the Dataset

The main functionality covers a wide range of banking-related queries, including customer regulations, asset appraisal procedures, preferential loan programs, and other financial services offered by the institution. The system is restricted in scope to ACB's internal regulations and official guidelines, ensuring that responses remain consistent with the bank's policies and compliant with regulatory requirements. ACB's staffs can interact with the chatbot to clarify procedural steps, explore eligibility conditions for specific loan products, or review applicable customer service rules. By focusing exclusively on ACB-specific knowledge, the chatbot provides structured and reliable responses that improve transparency and accessibility of banking information for both customers and staff. This specialized scope also lays the foundation for integrating retrieval-based techniques to ensure that the chatbot consistently delivers precise and regulation-aligned answers.

3.1.2 Portal Dataset

The vector database used about 1.400 PDF documents with total more than 30.000+ number of pages from portal system of ACB. Those files are mainly currently effective official dispatch. They cover wide range of ACB regulatory scope, including:

- Guidance for loan/deposit savings program.
- Guidance for valuation asset.
- Regulations for individual/company customers.
- IT-related regulations.
- Discount program.
- Collaborative program with partners.

- So on... .

3.2 Data processing stage

As mentioned above, data processing refer to the second stage where developer need to load data from those raw PDF files into vector database. There are two main phases that need to be done in this stage: parsing data and chunking data.

3.2.1 Parsing data

While PDF files are popular for their format that is friendly in the view and representation of information, utilizing or crawling the data directly from PDF seem to be very complex problem due to the dilemma: PDF format is optimized for information visualization, not data extraction. This make the processes of data retrieval, especially in those system that highly require the accuracy in data such as RAG pipelines, impossible to work directly on PDF-format data. This require the intermediate processing of data: parsing data. Parsing data refer to the process of extracting information from PDF files and convert into more structured format, may in Markdown or Json format. This ensure the text to be cleaned and support directly process of retrieval.

One of problem of parsing data is dealing with scan PDF file and unstructured format of data. Each PDF files can including images, table structure, multi-row format of texts, so on... , see Figure 12 more difficult to capture the information, which may leads to missing of data after parsing. Another problem is the capturing of wasted data such as header/footers information, which is very hard for auto detecting in post-parsing phases. This raise the challenges for parsing data process in which traditional approach using rule-based or text extraction work very poor.

LlamaParse is employed as the document parsing component within the LlamaIndex framework - provider of cloud-based services, enabling structured ingestion of heterogeneous data sources into the chatbot system. Its primary role is to preprocess raw documents—such as PDFs, Word files, or HTML pages—into well-defined nodes enriched with metadata, which can then be indexed and retrieved efficiently in downstream RAG pipelines. By standardizing unstructured content into a consistent format, LlamaParser ensures that the embedding and retrieval processes operate on clean and semantically meaningful units of information. In this project, LlamaParser was leveraged to transform portal PDF materials into structured representations that can be stored in Milvus and later retrieved by the LangChain–LangGraph workflow. As part of the broader LlamaIndex ecosystem, it provides a modular and extensible layer that abstracts away low-level parsing details, while maintaining compatibility with various storage backends and vector databases. This integration significantly improves the reliability of knowledge ingestion and the overall quality of responses generated by the Chatbot.

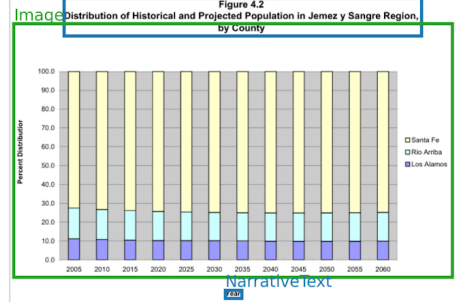
In my work, I've parsed document using LlamaParse with agentic mode, combine with OCR model. The prompt for summarizing images and skip header and footer information are also included to avoid the wasted of data being chunked and vectorized. All images object are prompted to be represented in side a couple of tags: <IMAGE.START> and <IMAGE.END>, which will be processed further in chunking step. The parsing happen in two phases:

- **Phase 1:** Extract raw text from PDF documents. OCR mode are integrated for maintaining the overall structure of whole PDF files.

NarrativeText

net-migration across the projection period. Differential in-migration of individuals over 25 years of age will result in dramatic aging of the Jemez y Sangre Region population between 2005 and 2060. The proportion of persons over 65 years of age will more than double—from 9.7 percent in 2005 to 24.2 percent in 2060 (Table 4.6). The result is likely a decrease in average household size over the projection period.

NarrativeText



Title

Table 4.5
Projected Population and Annual Average Growth Rate: July 1, 2000 to July 1, 2060
Jemez y Sangre Region

Jemez y Sangre Region														
Projection Year	Population				Population Share %				Projection Year	Annual Growth Rate %				
	Region	Los Alamos	Rio Arriba	Santa Fe	Region	Los Alamos	Rio Arriba	Santa Fe		Region	Los Alamos	Rio Arriba	Santa Fe	
2000	163,231	18,343	28,068	116,821	100	11.2	17.2	71.6	2000 - 05	1.81	1.59	0.87	2.06	
2005	178,665	19,864	29,318	129,483	100	11.1	16.4	72.5		0.92	0.27	0.40	1.13	
2010	187,020	20,129	29,912	136,979	100	10.8	16.0	73.2	2010 - 15	0.80	0.12	0.46	0.97	
2015	194,637	20,252	30,609	143,776	100	10.4	15.7	73.9		2015 - 20	0.68	0.25	0.39	0.80
2020	201,354	20,503	31,215	149,635	100	10.2	15.5	74.3	2020 - 25	0.47	0.37	0.15	0.55	
2025	206,141	20,880	31,457	153,804	100	10.1	15.3	74.6		2025 - 30	0.29	0.26	0.02	0.35
2030	209,130	21,158	31,495	156,477	100	10.1	15.1	74.8	2030 - 35	0.20	0.00	0.08	0.25	
2035	211,216	21,157	31,618	158,440	100	10.0	15.0	75.0		2035 - 40	0.14	-0.15	0.16	0.17
2040	212,669	21,004	31,871	159,794	100	9.9	15.0	75.1	2040 - 45	0.09	-0.08	0.17	0.09	
2045	213,585	20,915	32,142	160,528	100	9.8	15.0	75.2		2045 - 50	0.07	0.05	0.14	0.06
2050	214,330	20,971	32,373	160,986	100	9.8	15.1	75.1	2050 - 55	0.08	0.18	0.16	0.05	
2055	215,205	21,164	32,640	161,402	100	9.8	15.2	75.0		2055 - 60	0.14	0.32	0.28	0.09
2060	216,756	21,505	33,105	162,146	100	9.9	15.3	74.8						

Source: Bureau of Business and Economic Research, University of New Mexico

Source: Bureau of Business and Economic Research, University of New Mexico

55

Figure 12: Example of unstructured PDF files, which make the traditional parsing pipeline difficult to capture and represent data.



LlamaIndex

Figure 13: Llama Index ecosystem

- **Phase 2:** Use agentic LLM model for formatting and representing raw text into structured format PDF files.

The result of parsing stages are those markdown files that contain data for chunking.

3.2.2 Chunking & Vectorizing

In the next steps, from long Markdown files, I need to setup a strategy for break down them into small-enough and meaningful chunks, where each chunk will be a unit for storing/retrieving



in vector database. The basic introduction to chunking strategies is introduced in Section 2.2.2, you may want to review those foundation theory. In this project, I've developed an algorithm for chunking data, based on the **Hierarchical Chunking** strategy, with advanced processing to deal with table data and image data and capturing the sections title of each chunk while ensuring the length of each chunk. The working flow of chunking process is as followed:

1. Skipping image data, which is already formatted in form: `<IMAGE_START>` image description `<IMAGE_END>`.
2. Hierarchy detection of Markdown file, by travel whole while and detect the start of new section with `r"#{1,6}"` pattern.
3. Detect table of content by detecting vertical bar: `"|"`, I've recorded the header of table and saved for metadata detection.
4. Split those chunk that are too long in the same section with a predefined overlap ration.
5. Merge those chunk that are too short and in the same hierarchical level with same parent section.

Finally, all of the chunks are feeding into Milvus vector database (reader may found more detailed about Milvus in Section 4.2.4). About the semantic searching, the final similarity score used as weighted sum of COSINE similarity and BM25 score, which is an enhanced version of TF-IDF algorithm that I've learned in NLP course (CO3085). BM25 score normalize the score for each inverse document length term by total size of document length, to avoid the dominance of very long document as in TF-IDF. The weighted coefficient are 0.85 : 0.15 for COSINE and BM25.

4 Personal Project - Medical Chatbot

Conducting theoretical knowledge and experience observed from my instructors at company, I've built a simple Chatbot system to apply my knowledge in real-world application with usecase in medical field. From knowledge in fields of LLMs, RAG Chatbot and some basic knowledge in Backend design and utilizing of Chainlit - an UI application support interface for chatting with Chatbot, have been applied in my project, as shown in figure 14. The limitation in this project is that I have not yet integrate engineering tools for monitoring and evaluating performance of my application, such as: evaluate the performance of RAG retrieval data or evaluate on reranker result and the throughput of application.

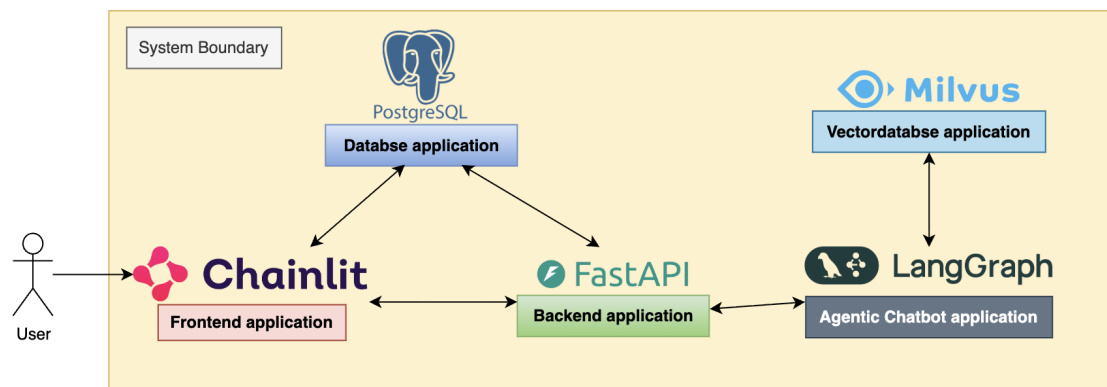


Figure 14: Micro-service architecture of Chatbot system.

The rest of this section is organized as followed: subsection 4.1 introduce about topic covered by this Chatbot, subsection 4.2 go in detailed about implementation of application using micro-service architecture. Finally, source of working is released in subsection 4.3.

4.1 Overview about Chatbot

The developed chatbot is designed as a specialized Medical AI Assistant with the primary objective of supporting users in retrieving evidence-based information across a wide range of medical specialties.

4.1.1 Covered Topic by the Dataset

The covered domains include genetic and rare diseases, growth hormone disorders, diabetes and metabolic conditions, neurological disorders and stroke, cancer, cardiovascular and respiratory diseases, kidney and digestive disorders, senior health, as well as public health topics related to disease prevention. Users may interact with the chatbot through diverse forms of questions, such as disease definitions ("What is/are..."), diagnostic procedures ("How..."), inheritance or risk factor queries ("Is..."), as well as treatment, prognosis, and risk assessment inquiries. In addition to domain coverage, the chatbot emphasizes delivering clear explanations of complex medical concepts using accurate terminology while remaining accessible to non-expert users.

Chatbot can understand the history context and can be used for predicting the disease from given list of symptoms. In the future, I am intended to add Chain-of-thoughts ability to Chatbot that

can handle more complex and professional medical advices for user.

4.1.2 Medical Q&A Dataset

The source of dataset is MedQuAD dataset - Medical Question Answering Dataset, from kaggle, which can be found in link: [Kaggle dataset](#). MedQuAD includes 47,457 medical question-answer pairs created from 12 NIH websites (e.g. cancer.gov, niddk.nih.gov, GARD, MedlinePlus Health Topics). The collection covers 37 question types (e.g. Treatment, Diagnosis, Side Effects) associated with diseases, drugs and other medical entities such as tests.

We use directly "Answer" data of each data for chunking. Where the study of tokens distribution are shown in Figure 15. This frequency of length distribution is suitable enough for storing chunks, so no need other processing step are introduced.

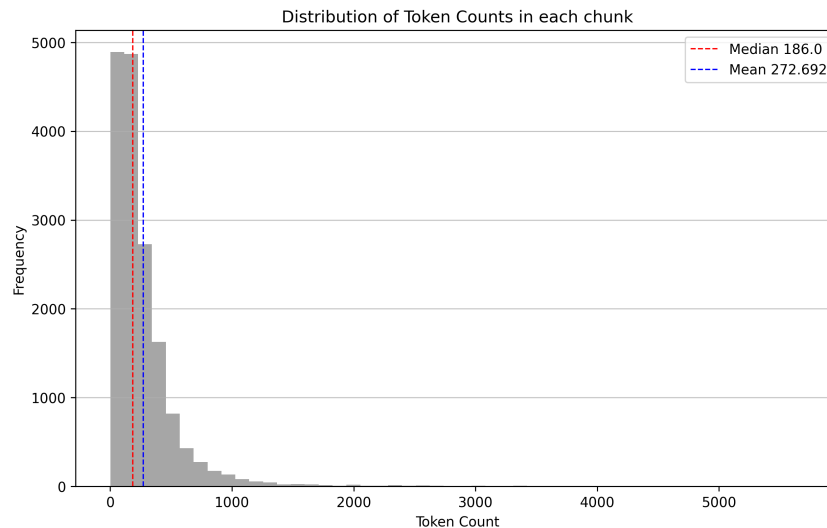


Figure 15: Histogram of tokens's frequency in each chunk

4.2 System Design

Inside the Chatbot system, I've implemented using microservice architecture along with cloud api service from ChatGPT for processing LLM-based steps. The detailed component views of application is shown in Figure 16.

In this section, I will present about overview of those architectures and try to state about the way that I utilized them in my Chatbot application, including: Chainlit, FastAPI, Milvus, LangGraph and LangChain. Finally, a brief review about multi-agents of this application is also introduced.

Table 2: List of checkpoints LLM models used in my application.

Task	Checkpoint	Provider
Text embedding	text-embedding-3-small	OpenAI Provider
Causal model	gpt-4o-mini	OpenAI Provider
Reranker	BAAI/bge-reranker-v2-m3	Local computer

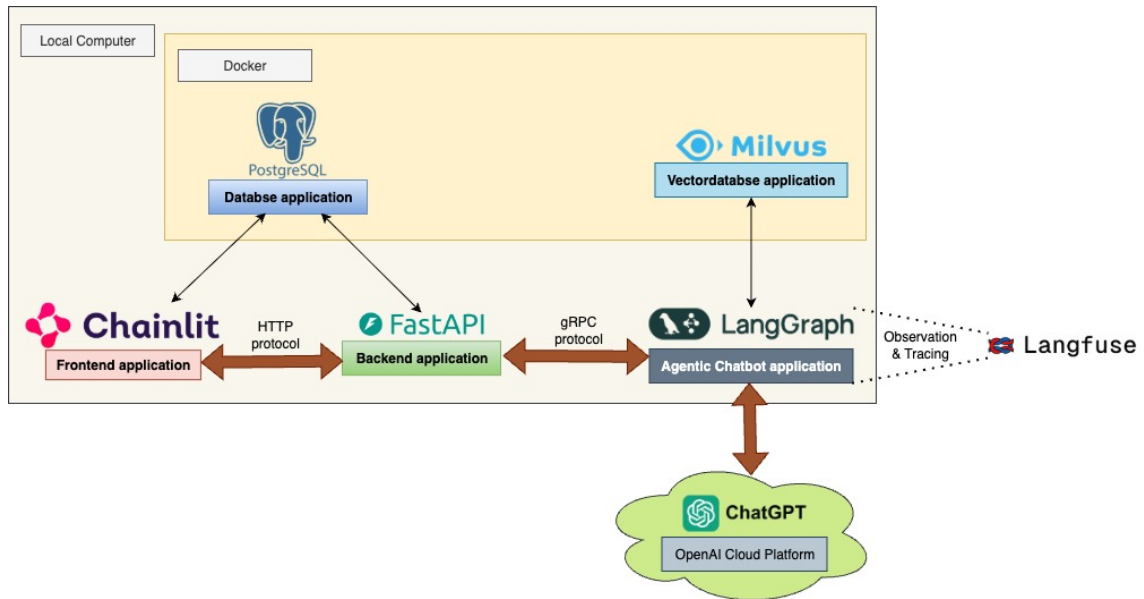


Figure 16: A components view of Chatbot application architecture

4.2.1 Sequence diagram

Before introducing detailed in each component and their technology, I would like to introduce about the general working flow of the main usecase of Chatbot: chat usecase. See Figure 17. This diagram present the interaction of all components in main usecase: chat and show the working and communication flow between components.

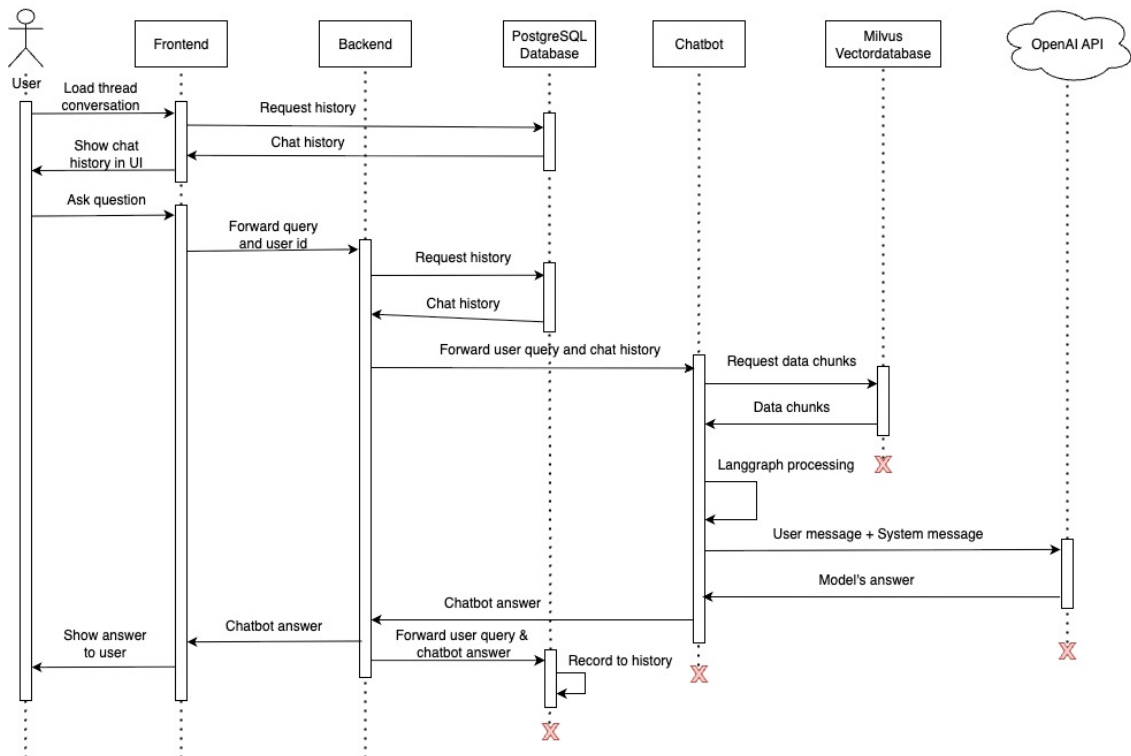


Figure 17: Sequence Diagram of chat usecase.

4.2.2 Chainlit Application

Chainlit is used to quickly build user-friendly Python-based chat applications, particularly for those interacting with large language models (LLMs). It simplifies the development process by providing a pre-built user interface and event-driven architecture, allowing developers to focus on the core logic of their application. Chainlit also offers features like data persistence, authentication, and integrations with popular LLM frameworks like Langchain.

Chainlit also provide flexibility code for integrating UI into micro-service application. This allow the combination with HTTP implementation that connect to Backend FastAPI. Some of the illustration for my Chatbot application are shown in Figure 19 and Figure 18.

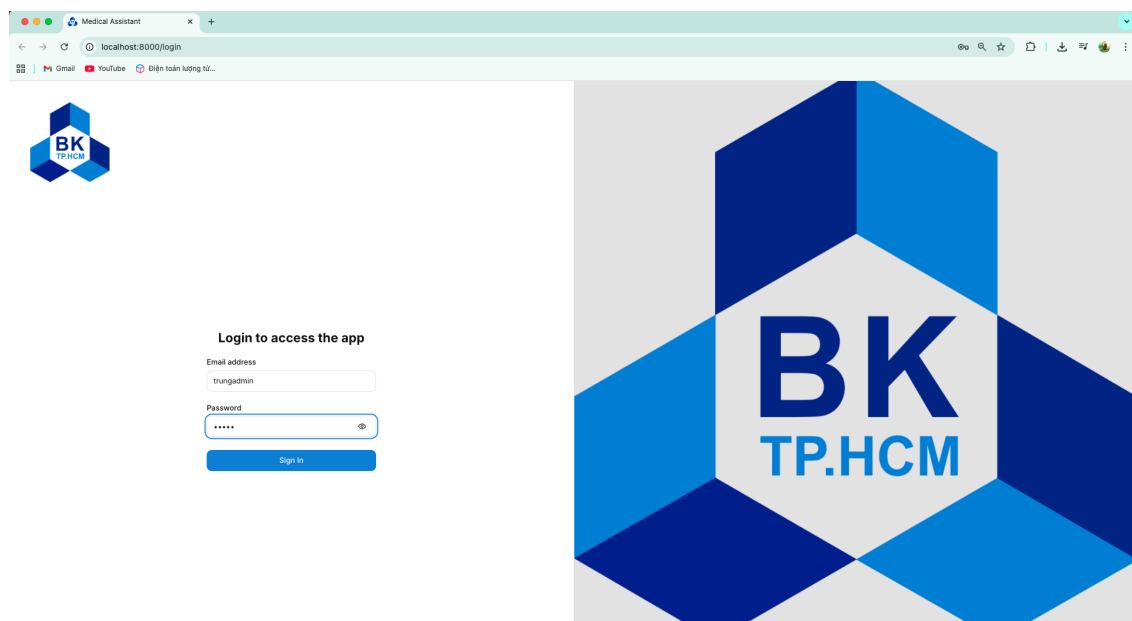


Figure 18: Chainlit UI application: Login Page. This page has been customized in my Chatbot application

4.2.3 FastAPI Backend

FastAPI is adopted as the backend application of the system, acting as the central processing layer that coordinates communication between the frontend interface, the database, and the agentic chatbot logic. Implemented as a high-performance web framework based on Python, FastAPI enables the system to handle user requests with low latency and strong scalability. In this project, FastAPI exposes RESTful endpoints through the HTTP protocol to receive queries from the Chainlit frontend, manages data exchange with PostgreSQL for conversation storage, and forwards semantic retrieval or generation tasks to the LangGraph-based chatbot module using gRPC protocol. Its asynchronous design and automatic data validation mechanisms provide a reliable foundation for integrating microservices, ensuring that the chatbot maintains robustness and efficiency under different usage scenarios.

All services provided in this application:

- **Health Check:** used for docker container to periodically check health of backend docker container.

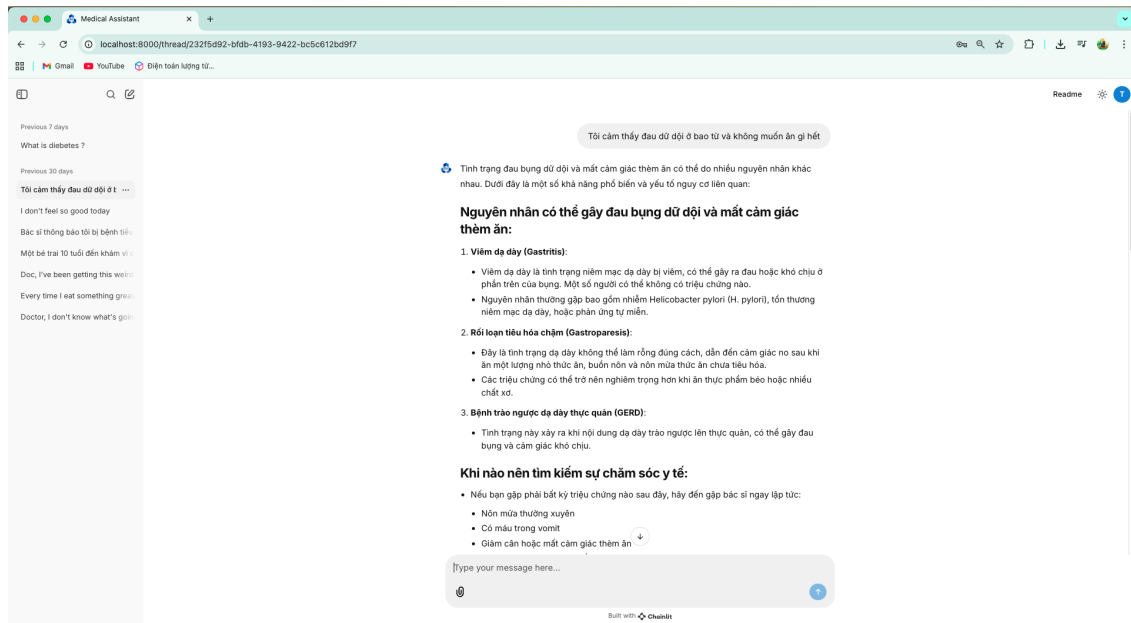


Figure 19: Chainlit UI application: Conversation Page

- **Authentication:** before using application, user must login using their username and password that is registered in database.
- **History management:** save and manage history, thread of conversation of user.
- **Chat serving:** connect to Chatbot via gRPC protocol and use streaming response for improve the experience of user.

4.2.4 Milvus vectordatabase

Milvus is utilized as the vector database application in the system, serving as the core storage and retrieval engine for embedding-based information. Within the chatbot pipeline, text data from the medical Q&A dataset and user queries are transformed into high-dimensional vector representations, which are efficiently indexed and queried through Milvus. This enables semantic similarity search, allowing the chatbot to retrieve contextually relevant information rather than relying solely on keyword matching. By leveraging Milvus, the system ensures scalability in handling large volumes of embeddings and provides optimized query performance through approximate nearest neighbor (ANN) search algorithms. Integrated within the Dockerized local environment, Milvus acts as the backbone of the Retrieval-Augmented Generation (RAG) process, directly supporting the accuracy and responsiveness of the chatbot application.

For searching in vectordatabase, I utilized only semantic search with COSINE metrics.

4.2.5 Langgraph & Langchain Framework

LangChain is employed as the foundational framework that orchestrates the interaction between large language models, external data sources, and tool components within the chatbot system. Its modular design provides abstractions for prompt management, retrieval pipelines, memory handling, and agent-based reasoning, which greatly simplifies the construction of complex conversational workflows. In this project, LangChain is responsible for integrating the RAG process:

embedding user queries, performing semantic search over the Milvus vector database, reranking retrieved results, and passing the most relevant context to ChatGPT for response generation. By leveraging LangChain, the system benefits from a flexible architecture that supports composability and extensibility, enabling the chatbot to adapt to new functionalities such as query rewriting, multi-step reasoning, or integration with additional APIs in the future.

LangGraph is introduced into the system as an extension of the LangChain ecosystem, providing a graph-based framework for designing and executing complex agent workflows. While LangChain offers the fundamental abstractions for prompt templates, retrieval pipelines, and memory, LangGraph builds on top of these concepts by representing the workflow as a directed graph where each node corresponds to a computation step (e.g., retrieval, reranking, or generation) and edges define the flow of data. This design allows for more transparent visualization, modularity, and control of multi-step reasoning processes. In the chatbot system, LangGraph is applied to structure the RAG pipeline into explicit nodes, making it easier to trace the intermediate results and to integrate observability tools such as Langfuse. By leveraging LangGraph as a higher-level orchestration layer, the system gains flexibility in extending to multi-agent interactions and adaptive workflows, while still retaining full compatibility with the underlying LangChain components. Figure 20 shows Langgraph Graph that has been designed in this application.

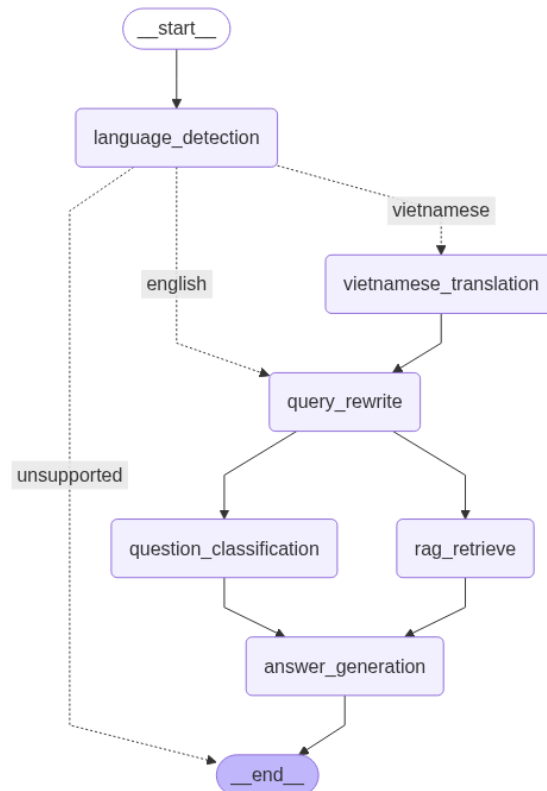


Figure 20: *Langgraph graph utilized in my Chatbot application.*

Langgraph graph mechanism implemented in this Chatbot application:

- **Language detection node:** detect the main language that user is using. In this application, Chatbot only allows using either English or Vietnamese.



- **Vietnamese translation node:** translate all of Vietnamese text into English. Because I am using embedding model **text-embedding-3-small** which do not support well Vietnamese. In experiment, translation make the retrieval method perform more accurately.
- **Query rewrite node:** perform rewrite the answer using history context to achieve standalone query and also to filter out those wasted words.
- **RAG retrieval node:** perform COSINE similarity for retrieval most relevant chunks and also rerank them.
- **Question classification node:** classify the type of question to either types: education driven, medical advice, symptoms to disease prediction or specific inquiry. Hence model can adjust the tone of answer.
- **Answer generation:** generate answer of models from the user's question and context return by RAG retrieval node.

4.2.6 Langfuse

Langfuse is incorporated into the system as an observability and tracing tool, enabling monitoring of the chatbot's behavior and evaluation of its performance across different components. It provides detailed logs of requests, responses, and intermediate steps within the agentic workflow, which helps in diagnosing errors, analyzing latency, and tracking data flow between services. In this project, Langfuse is connected to the LangGraph-based chatbot module to capture structured traces of retrieval results, reranker outputs, and generated responses. This observability layer not only facilitates debugging and performance optimization but also establishes the foundation for future systematic evaluation of retrieval accuracy, reranking effectiveness, and end-to-end throughput. By integrating Langfuse, the system benefits from improved transparency and maintainability, supporting iterative refinement of the chatbot's capabilities.

In my application, I utilized Langfuse for observing trace and manage version of prompts. See Figure 21 for example of tracing.

Timestamp ▼	Name	Input	Output	Observation Levels	Latency	Tokens	Total Cost
2025-08-18 17:14:35	answer_generation_node	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	"Diarrhea can be caused by various factors, including infections, food intolerances, stress, or underlying medical conditions. It's important to consult a healthcare professional for a proper diagnosis and treatment plan."	1	7.63s		
2025-08-18 17:14:28	RAGRetrievalNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	"Context: [\"question\": \"What causes What I need to know about D...\"]	1	6.82s		
2025-08-18 17:14:28	QuestionClassificationNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	("question_type": "MEDICAL_GUIDANCE")	1	1.29s		
2025-08-18 17:14:27	QueryRewriteNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"history": [{"role": "user", "content": "What could be the cause of experiencing daily diarrhea every morning?"}], "question": "What could be the cause of experiencing daily diarrhea every morning?"})	"What could be the cause of experiencing daily diarrhea every morning?"	1	1.36s		
2025-08-18 17:14:26	VietnameseTranslationNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	"What could be the cause of experiencing daily diarrhea every morning?"	1	0.89s		
2025-08-18 17:14:25	DetectLanguageNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	("user_language": "Vietnamese")	1	0.90s		
2025-08-18 17:14:25	LangGraph	("question": "What could be the cause of experiencing daily diarrhea every morning?"})	("history": [{"role": "user", "content": "What could be the cause of experiencing daily diarrhea every morning?"}])	14	17.82s	7,221 + 642 (7,863) ○	\$0.001468 ○
2025-08-18 17:13:31	answer_generation_node	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	"Diarrhea can be caused by various factors, including infections, food intolerances, stress, or underlying medical conditions. It's important to consult a healthcare professional for a proper diagnosis and treatment plan."	1	8.51s		
2025-08-18 17:13:20	QuestionClassificationNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	("question_type": "MEDICAL_GUIDANCE")	1	1.21s		
2025-08-18 17:13:20	RAGRetrievalNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	"Context: [\"answer\": \"Diarrhea means that you have loose, watery...\"]	1	9.76s		
2025-08-18 17:13:19	QueryRewriteNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"history": [{"role": "user", "content": "What could be the cause of experiencing daily diarrhea every morning?"}], "question": "What could be the cause of experiencing daily diarrhea every morning?"})	"What could be the cause of experiencing daily diarrhea every morning?"	1	1.29s		
2025-08-18 17:13:18	VietnameseTranslationNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	"What could be the cause of experiencing daily diarrhea every morning?"	1	0.84s		
2025-08-18 17:13:17	DetectLanguageNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	("user_language": "Vietnamese")	1	1.71s		
2025-08-18 17:13:17	LangGraph	("question": "What could be the cause of experiencing daily diarrhea every morning?"})	("history": [{"role": "user", "content": "What could be the cause of experiencing daily diarrhea every morning?"}])	14	22.71s	3,370 + 590 (3,960) ○	\$0.000859 ○
2025-08-18 17:13:17	DetectLanguageNode	("args": [{"system": "You are a specialized medical AI assistant trained to provide accurate medical advice and diagnosis based on the latest research and clinical guidelines."}], "kwargs": {"question": "What could be the cause of experiencing daily diarrhea every morning?"})	("user_language": "Vietnamese")	1	1.00s		
2025-08-18 17:13:149	LangGraph	("question": "What could be the cause of experiencing daily diarrhea every morning?"})		3	1.01s	305 + 0 (305) ○	\$0.000045 ○

Figure 21: Example of tracing history in Langfuse

4.3 Sources

In this section, I shared all of my sources of working including: usecases of using Chatbot application, slide for presenting in my internship and the github repository of my source code:



- [Google Drive](#): contains some video for example usecases and slides of presenting.
- [Github Repo](#): github repository for coding. Those commit code reflect my progression of working on this code.



5 Conclusion

During internship in Asia Commercial Bank, working at a very professional environment of big company give me a chance to not only learn new specialization practical knowledge, but also a chance for me to learn and develop new set of soft skills that are very valuable in my upcoming journey of my career. Rather than regarding this as a forced activity of university time, from my perspective, this internship time give me an very precious experience to my set of skills and help me to understand more about what is needed by the AI labor market. On the other hand, this summer trip in third year also bring for me a very important aspect - relationship and networks, not only in the working scope but as a friend, a tutorial who inspire me in my career way. From the perspective of knowledge, I have a chance to learn new thing and can apply them via two project: ACB Chatbot Portal (Section 3) and Medical Chatbot (Section 4). These two project are mutual to each other on the ground that I can approach and learn different progress of building a Chatbot's workflow: from processing data to design a project architecture and also learn the DevOps works.

In short, I want to express my appreciate toward my interned instructors, Mr.Le Huy Hoang for his guidance and follow up me all the time of my internship, as well as Mr.Nguyen Manh Trung and Mr.Phan Minh Chien for their support me in my internship time. I sincerely wish the AI team continued growth and great success in the future.

Student watermark

Tran Quoc Trung

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [3] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [4] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [5] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [7] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [8] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2025.